

Module 7: Sequence Models for NEC Gating

A single merged treatment of Goodfellow Ch. 10 and Olah’s

“Understanding LSTM Networks,” ending inside the NEC code

Prepared for Tom Koreslesjak · companion to `NEC_thesis_syllabus.md`

June 2026

Abstract

The last architectural piece of the thesis: the recurrent encoder that turns a window of market history into the vector the gate routes on. The chapter derives backpropagation through time as Module 6’s backprop with weights tied across time, proves the vanishing/exploding-gradient theorem as the third and final appearance of this book’s geometric-amplification motif, and dissects the GRU and LSTM gates as engineered gradient highways — per-coordinate, input-dependent forgetting rates with readable memory timescales. It closes by reading `nec_hybrid_explained.py` line by line at the level of mathematics rather than code: every tensor shape, every design defect, and the precise textbook section that diagnosed each one. By the end, nothing in the existing NEC is mysterious, and every change the corrected model makes has a derivation behind it.

Contents

7.1 The encoder’s job: from a window of history to one vector	2
7.2 Backpropagation through time	3
7.3 The vanishing/exploding-gradient theorem	4
7.4 Gates: engineered gradient highways	5
7.4.1 GRU	5
7.4.2 LSTM	6
7.5 Hidden states, and why financial sequence models overfit	7
7.6 Capstone: reading the NEC code, line by line	8
7.7 Exercises	9

How this chapter was assembled (and what was cut)

Audit findings first, because this module’s reading list needed surgery. (i) The syllabus assigns “Prince Ch. 12.1–12.6” for RNNs, GRU, and LSTM. Prince’s *Understanding Deep Learning* Chapter 12 is **Transformers** — verified against the local copy — and the book contains *no* recurrent-networks chapter at all (the author omitted RNNs deliberately). The assigned “Prince Problems 12.1–12.2” are self-attention problems, irrelevant to this module’s topics; Exercise 7.7 substitutes. (ii) The syllabus assigns “Goodfellow Ch. 10, Exercises 10.1–10.2”; as established in Module 6’s audit, Goodfellow contains no exercises; Exercise 7.8 substitutes. The chapter’s real sources are therefore Goodfellow Ch. 10 (which covers this material thoroughly) and Olah’s essay; Prince Ch. 12 becomes relevant only if transformer baselines are added to the thesis later.

Cuts from Goodfellow Ch. 10, with reasons:

- *Encoder–decoder / sequence-to-sequence machinery* (§10.4), *recursive nets* (§10.6), *echo-state networks*. The NEC consumes a sequence *encoder* — one vector out — not a translation system; seq2seq survives as the one-paragraph contrast the topic list asks for (§7.1).
- *Teacher forcing* (§10.2.1). A training device for output-feedback models; an encoder has no outputs to feed back. One sentence where relevant.
- *Bidirectional RNNs* (§10.3). Not cut — *converted*: in a prediction pipeline bidirectionality is a leakage question, and it becomes Exercise 7.8, whose answer is more subtle than “illegal.”
- *Gradient clipping mechanics*. Done in Module 6 §6.6; here it gets its second justification (mechanical explosion) and a cross-reference.
- *Leaky units and skip connections through time* (§10.9). One sentence as the linear precursor of gates.

Notation

Sequences run $t = 1, \dots, T$ with inputs $x_t \in \mathbb{R}^d$, hidden states $h_t \in \mathbb{R}^n$, and (for LSTM) cell states $c_t \in \mathbb{R}^n$. $\sigma(\cdot)$ is the elementwise logistic sigmoid, $\tanh(\cdot)$ elementwise, $\odot$ elementwise product. Recurrent Jacobians are $J_t = \partial h_t / \partial h_{t-1}$ ($n \times n$). The encoder’s deliverable is h_T (or $[h_T; c_T]$), the fixed-length summary handed to the gate. Module and equation cross-references follow the established ledger: (B.·) Foundations B, (C.·) Foundations C, (4.·)/(5.·)/(6.·) Modules 4–6.

7.1 The encoder’s job: from a window of history to one vector

The NEC’s data interface (syllabus §3) is a tensor of shape (batch, T, d): for each asset-date, a trailing window of T daily feature vectors. The encoder’s job is a map

$$(x_1, \dots, x_T) \mapsto h_T \in \mathbb{R}^n, \quad (7.1.1)$$

a *learned summary statistic* of the window, consumed here by the gate (and, optionally, by experts). Hold two reference points while learning the machinery. First, you already use fixed encoders daily: a 20-day realized volatility is (7.1.1) with a hand-chosen quadratic form; an EMA of returns is (7.1.1) with a hand-chosen exponential kernel (Exercise 7.7 makes this exact: an EMA *is* a one-dimensional linear RNN). The recurrent encoder’s promise is only that the kernel and the nonlinearity are learned rather than chosen — a promise that must beat the hand-chosen versions out of sample to justify its variance (§7.5). Second, the contrast the topic list requests: *sequence-to-sequence* models (translation, transcription) emit an output *per step* or decode a new sequence from the summary; they need alignment machinery, teacher forcing, and decoder stacks. An *encoder-only* use case needs none of it — which is why this chapter is shorter than Goodfellow Ch. 10 and why several of its sections were cuttable.

The architecture: a *recurrent* network maintains a state and applies one shared transition repeatedly,

$$h_t = \tanh(W h_{t-1} + U x_t + b), \quad h_0 = \mathbf{0}, \quad (7.1.2)$$

the *vanilla RNN*. Read it as a discrete-time driven dynamical system: W is the autonomous dynamics (what the state does on its own), $U x_t$ the forcing, $\tanh$ the saturation that keeps the state bounded. One transition, applied T times: the parameter count is independent of T , which is the entire point — and, as the next section shows, also the entire problem.

7.2 Backpropagation through time

Unroll (7.1.2): the computation from $x_{1:T}$ to a loss $\mathcal{L}$ on h_T is a T -layer feedforward network whose layers *share* the parameters (W, U, b) . Module 6’s backprop machinery (the boxed recursion 6.4.1) therefore applies verbatim, with one new feature: because every “layer” holds the same W , the total gradient *sums* the per-layer contributions. Running the chain rule along the unrolled graph (Exercise 7.1 executes it for $T = 3$ explicitly, then by induction):

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^T \underbrace{\frac{\partial \mathcal{L}}{\partial h_T}}_{\text{from the head}} \underbrace{\left(\prod_{s=t+1}^T J_s \right)}_{\text{through time}} \underbrace{\frac{\partial^+ h_t}{\partial W}}_{\text{local, at step } t}, \quad J_s = \frac{\partial h_s}{\partial h_{s-1}} = \text{diag}(1 - h_s^{\odot 2}) W, \quad (7.2.1)$$

where ∂^+ denotes the direct (single-step) dependence and the Jacobian uses $\tanh' = 1 - \tanh^2$. This is *backpropagation through time*: Module 6’s boxed recursion with the VJP running along the time axis, sensitivities $\delta_t = J_{t+1}^\top \delta_{t+1}$ flowing right to left. Memory cost is Foundations C, Exercise C.7(b) made painful: all T hidden states must be stored, so memory grows linearly in window length — the practical reason for *truncated* BPTT, which backpropagates only $k < T$ steps from each loss and treats older states as constants. Truncation is not free: it deletes exactly the terms $t \leq T - k$ from (7.2.1), biasing the gradient against long-range structure; Exercise 7.6 quantifies the bias and uncovers its cruel punchline — truncation is harmless precisely when those terms were vanishing anyway.

Checkpoint exercise

Exercise 7.1 [Syllabus: BPTT for three steps, then T]. [★★★★ Core] Take the vanilla RNN (7.1.2) with $T = 3$ and a loss $\mathcal{L}$ on $\mathbf{h}_3$ only. (a) By explicit chain rule on the unrolled graph (no formula-quoting), derive $\partial\mathcal{L}/\partial\mathbf{W}$ as the sum of three terms — the direct contribution at $t = 3$, the one-hop contribution from $t = 2$ through $\mathbf{J}_3$, and the two-hop contribution from $t = 1$ through $\mathbf{J}_3\mathbf{J}_2$ — with $\mathbf{J}_s = \text{diag}(1 - \mathbf{h}_s^{\odot 2})\mathbf{W}$. Keep the $\partial^+\mathbf{h}_t/\partial\mathbf{W}$ outer-product structure explicit (Foundations C, Example C.6.1: sensitivity times input, here input = $\mathbf{h}_{t-1}$). (b) Generalize to arbitrary T by induction, recovering (7.2.1), and derive the same result for $\partial\mathcal{L}/\partial\mathbf{U}$ (what replaces $\mathbf{h}_{t-1}$ in the outer product?). (c) Two structural observations, one sentence each: why does weight tying turn the gradient into a sum over time (contrast Module 6, where each layer’s weight got one term)? And which intermediate quantities must be cached, confirming the linear-in- T memory claim of §7.2?

7.3 The vanishing/exploding-gradient theorem

Everything interesting in (7.2.1) sits in the product $\prod_{s=t+1}^T \mathbf{J}_s$ — a product of $T - t$ Jacobians, and the third time this book has met a geometric amplification: gradient-descent modes contract as $(1 - \eta\lambda)^t$ (Foundations C §C.2), depth scales variance as $(n\sigma_W^2)^L$ (C §C.7, Module 6 §6.5), and now temporal credit assignment scales as the spectral radius to the power of the time gap. One tool makes the statement clean, recorded as a fact:

Lemma 7.3.1 (Gelfand’s formula, stated). For any square matrix $\mathbf{J}$ and any matrix norm, $\lim_{k \rightarrow \infty} \|\mathbf{J}^k\|^{1/k} = \rho(\mathbf{J})$, the spectral radius (largest eigenvalue magnitude). Hence $\|\mathbf{J}^k\|$ grows or decays geometrically at asymptotic rate $\rho(\mathbf{J})^k$, whatever the norm says for small k .

Theorem 7.3.2 (Vanishing/exploding gradients). Suppose the recurrent Jacobians share the bound-relevant structure $\mathbf{J}_s \approx \mathbf{J}$ with spectral radius ρ (the constant-Jacobian idealization; Exercise 7.2 states the honest varying- $\mathbf{J}_s$ version with $\sup_s \|\mathbf{J}_s\| \leq \rho$). Then the gradient contribution of step t to a loss at T scales as ρ^{T-t} : for $\rho < 1$ it vanishes geometrically — early inputs become invisible to learning — and for $\rho > 1$ it explodes geometrically — gradients are dominated by rare amplification events and training destabilizes.

For the vanilla RNN the structure is worse than neutral: $\mathbf{J}_s = \text{diag}(1 - \mathbf{h}_s^{\odot 2})\mathbf{W}$, and the diagonal factor is ≤ 1 everywhere and $\ll 1$ wherever $\tanh$ saturates — so even with $\mathbf{W}$ tuned to $\rho(\mathbf{W}) \approx 1$, saturation drags the effective radius below one, and vanishing is the *default*, not the accident. The two failure modes get asymmetric remedies, both already in your kit or about to be: explosion is occasional and mechanical, handled by the norm clipping of Module 6 §6.6 (this is that section’s promised second justification); vanishing is structural and cannot be clipped away — it requires changing the architecture so that some path through time has Jacobian $\approx \mathbf{I}$ by *construction*. Linear self-loops (“leaky units”) are the primitive version; gates, next, are the engineered one.

Finance reality check

Translate ρ into trading horizons: the effective memory of an encoder is $\tau \approx 1/\log(1/\rho)$ steps — at an effective $\rho = 0.9$, about ten days; at $\rho = 0.99$, about a hundred. The financial structures the NEC gate wants to see — volatility regimes persisting for weeks to months (Foundations B §B.7’s clustering) — sit at the long end, exactly where a vanilla RNN is congenitally blind. This, and not fashion, is why the existing NEC reaches for an LSTM and why the corrected model will keep *some* gated encoder: the architecture choice is a statement about which autocorrelation horizons the gate is allowed to learn from.

Checkpoint exercise

Exercise 7.2 [Syllabus: the vanishing/exploding theorem, honestly]. [★★★ Core] (a) *Constant-Jacobian case:* for $\mathbf{J}_s = \mathbf{J}$ with spectral radius ρ , use Lemma 7.3.1 to show the step- t gradient contribution in (7.2.1) scales as ρ^{T-t} asymptotically, giving explosion for $\rho > 1$ and vanishing for $\rho < 1$. (b) *Varying Jacobians:* show that $\sup_s \|\mathbf{J}_s\|_2 \leq \bar{\rho} < 1$ forces $\|\prod_{s=t+1}^T \mathbf{J}_s\|_2 \leq \bar{\rho}^{T-t}$ (submultiplicativity — Foundations A norms), so vanishing needs only a uniform bound; explain why the symmetric claim for explosion is not automatic (a product of matrices each with some eigenvalue > 1 need not blow up — give a 2×2 rotation-flavored counterexample or argue why directions matter). (c) For the vanilla RNN specifically, bound $\|\mathbf{J}_s\|_2 \leq \|\mathbf{W}\|_2 \cdot \max_i (1 - h_{s,i}^2)$ and conclude that $\tanh$ saturation strictly tightens the vanishing bound; where in state space is the network most amnesiac? (d) State the implication for financial sequences in two sentences, using the memory-horizon translation $\tau \approx 1/\log(1/\rho)$ of §7.3’s box.

7.4 Gates: engineered gradient highways

7.4.1 GRU

The gated recurrent unit replaces the monolithic transition (7.1.2) with a *convex-combination* update controlled by two learned, input-dependent gates:

$$\begin{aligned} \mathbf{r}_t &= \sigma(\mathbf{W}_r \mathbf{h}_{t-1} + \mathbf{U}_r \mathbf{x}_t + \mathbf{b}_r) && \text{(reset: how much past to consult)} \\ \mathbf{z}_t &= \sigma(\mathbf{W}_z \mathbf{h}_{t-1} + \mathbf{U}_z \mathbf{x}_t + \mathbf{b}_z) && \text{(update: how much to rewrite)} \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{U}_h \mathbf{x}_t + \mathbf{b}_h) && \text{(candidate state)} \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t && \text{(interpolation).} \end{aligned} \tag{7.4.1}$$

The last line is the architectural insight: the new state is a per-coordinate *blend* of the old state and a proposal, not a wholesale nonlinear rewrite. Differentiate it: holding the gate paths aside,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \supseteq \text{diag}(1 - \mathbf{z}_t), \tag{7.4.2}$$

so wherever the update gate closes ($\mathbf{z} \rightarrow 0$), the Jacobian’s direct term approaches the *identity* — a path through time with $\rho = 1$ by construction, immune to Theorem 7.3.2’s decay, for exactly the coordinates the network elects to preserve. Exercise 7.3 derives the full Jacobian including the gate paths and confirms the suppression claim honestly.

Checkpoint exercise

Exercise 7.3 [Syllabus: the GRU’s gradient highway]. [★★★★ Core] (a) From the gate equations (7.4.1), assemble the update $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$ and compute the full Jacobian $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}$: the direct term $\text{diag}(1 - \mathbf{z}_t)$ plus the three indirect paths (through $\mathbf{z}_t$, through $\tilde{\mathbf{h}}_t$ ’s recurrent argument, and through $\mathbf{r}_t$). (b) Show that as $\mathbf{z}_t \rightarrow \mathbf{0}$ the indirect paths are suppressed along with the rewrite (which factors kill them?), so the Jacobian tends to the identity — the precise version of (7.4.2). (c) Interpret each gate in rolling-statistic language: what fixed setting of $(\mathbf{r}, \mathbf{z})$ reproduces (i) a plain EMA of the inputs, (ii) a frozen state that ignores new data entirely?

7.4.2 LSTM

The LSTM achieves the same end with an explicit *memory register*. Olah’s image is the right one: a cell state $\mathbf{c}_t$ rides a conveyor belt along time, touched only by elementwise gates; the hidden state $\mathbf{h}_t$ is a gated *readout* of the cell:

$$\begin{aligned} \mathbf{f}_t &= \sigma(\cdot), \quad \mathbf{i}_t = \sigma(\cdot), \quad \mathbf{o}_t = \sigma(\cdot), \quad \mathbf{g}_t = \tanh(\cdot) \quad (\text{forget, input, output, candidate}) \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t \quad (\text{the conveyor belt}) \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (\text{readout}), \end{aligned} \tag{7.4.3}$$

each gate a sigmoid of $(\mathbf{W}_\bullet \mathbf{h}_{t-1} + \mathbf{U}_\bullet \mathbf{x}_t + \mathbf{b}_\bullet)$ with its own parameters. The belt’s Jacobian, gates held aside, is

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \supseteq \text{diag}(\mathbf{f}_t) : \tag{7.4.4}$$

where the forget gate saturates open ($f \rightarrow 1$), the cell-state pathway is the identity — “LSTM gates protect gradient flow” as a one-line theorem (Exercise 7.4 does it properly, indirect paths and all, and explains the once-standard trick of initializing $\mathbf{b}_f$ positive so training *starts* with long memory). Note what is additive and what is multiplicative: information *enters* the cell additively ($+ \mathbf{i} \odot \mathbf{g}$), so writing does not erase — contrast the vanilla RNN, where every step rewrites the whole state through a squashing nonlinearity.

Read the gates with Foundations B eyes: a sigmoid gate is a two-state Boltzmann distribution (B §B.6.3) per coordinate — a soft binary decision, “keep or rewrite,” made continuously and differentially. A forget gate stuck at f implements an EMA with timescale $\tau \approx -1 / \log f$ (Exercise 7.7): *the LSTM is, among other things, a bank of learned, state-dependent EMAs*, which is precisely the language in which a quant already thinks about rolling features. And the conceptual rhyme with the thesis is worth saying aloud: sigmoid gates route information *within* a model across time; the NEC’s softmax gate routes samples *across* models — the same Boltzmann machinery (B §B.6) at two different scales, and the reason the syllabus titles this module “sequence models for NEC gating.”

GRU versus LSTM (Exercise 7.5 makes it quantitative): the GRU drops the separate cell state, merges forget/input into one update gate ($\mathbf{z}_t$ and $1 - \mathbf{z}_t$: writing *is* forgetting), and drops the output gate — three weight blocks per layer instead of four, $3n(n + d + 1)$ parameters versus $4n(n + d + 1)$. On small, noisy datasets the GRU is routinely as good or better — 25% fewer parameters is pure variance reduction (Foundations B, eq. B.5.1) with no demonstrated bias

penalty at this scale — which is why the syllabus’s own corrected-model sketches (the seq model in OP model/) already use GRUs, and why “replace the 8-layer LSTM with a 1–2-layer GRU” will appear in the capstone’s recommendations.

Checkpoint exercise

Exercise 7.4 [Syllabus: the LSTM’s conveyor belt]. [★★★★ Core] (a) From (7.4.3), compute $\partial c_t / \partial c_{t-1}$ in full: the direct term $\text{diag}(f_t)$ plus the indirect paths through the gates’ dependence on $h_{t-1} = o_{t-1} \odot \tanh(c_{t-1})$. (b) Show that with $f_t \rightarrow 1$ (and gates otherwise unsaturated) the cell pathway is approximately the identity, and explain why the additive write $+ i_t \odot g_t$ matters for this argument — what would a multiplicative write do to the highway? (c) Explain the classical practice of initializing the forget-gate bias b_f to $+1$ or $+2$ in terms of where σ saturates, and what it buys during early training. (d) What does the output gate o_t decouple from what? Give one financial example where a quantity worth remembering (in c) is not worth emitting (in h) at every step.

Exercise 7.5 [Syllabus: LSTM vs. GRU, quantitatively]. [★★★★ Core] (a) Count parameters per layer for input dimension d and hidden size n , biases included: show $\text{LSTM} = 4n(n + d + 1)$ and $\text{GRU} = 3n(n + d + 1)$, and identify exactly which structures the GRU drops (separate cell state; independent input gate, replaced by $1 - z$; output gate). (b) Evaluate both counts for the existing NEC encoder ($d = 1$ ostensibly, $n = 256$, 8 layers — note layers above the first have input dimension n) and for the proposed replacement ($n = 32$, 2 layers). What is the ratio? (c) Argue the small-data case for the GRU in bias–variance terms (Foundations B, eq. B.5.1), and state the honest caveat: what could the LSTM’s extra machinery buy that a GRU cannot, and under which data regime would you expect to detect it?

7.5 Hidden states, and why financial sequence models overfit

What is h_T ? Mechanically, a learned rolling statistic: a point in $\mathbb{R}^n$ summarizing the window through learned kernels and gates. Two interpretation tools keep it honest, both cheap. *Probe regressions*: regress each coordinate (or principal component — Foundations A) of h_T on the hand-crafted features you already trust (realized vol, momentum, drawdown); high R^2 means the encoder has rediscovered known features (reassuring, and deflating), low R^2 means it found something else (interesting, and suspicious — in that order). *Ablation against fixed encoders*: the OP pipeline’s rolling features *are* a hand-built (7.1.1); a learned encoder that cannot beat them out of sample, under Module 5’s protocol, is negative evidence with high credibility.

The deck is stacked toward overfitting, by an accounting you can now perform precisely. The effective sample is far smaller than the row count: adjacent windows share $T - 1$ days (Module 5 §5.4’s overlap channel, now inside the *features*), cross-sectional rows share the date’s market move (channel 3), and the usable signal is Foundations B’s single-digit-percent sliver. Against that, an 8-layer, 256-unit LSTM brings on the order of 4×10^6 parameters — Module 6 §6.2’s region-counting surplus pointed in exactly the wrong direction. The defensive checklist, every line a cross-reference: small hidden dimensions and shallow stacks (6.2); LayerNorm, never BatchNorm, inside sequence models (6.7 — and note the existing NEC violates this in its classifier head); dropout between recurrent layers only with care (naive i.i.d. dropout re-sampled each step

disrupts the very memory the gates protect; variational/locked dropout re-uses one mask per sequence — one sentence of awareness is enough here); norm clipping (6.6); early stopping on purged-validation rank-IC (6.8, 5.5); and the probe/ablation honesty tests above.

7.6 Capstone: reading the NEC code, line by line

With the machinery built, `OP model/nec/nec_hybrid_explained.py` reads in ten minutes. The shape flow, for input x of shape (B, T, d) :

Stage	Code	Output shape	Mathematics
Encoder	C_L: 8-layer LSTM, $n = 256$	out $(B, T, 256)$	eq. (7.4.3), stacked; all steps returned
Head select	out[:, -1, :] inside C_F	$(B, 256)$	take h_T of last layer
Head norm	BatchNorm1d	$(B, 256)$	eq. (6.7.1) across the <i>batch</i>
Gate scores	Linear(256, 3) + Softmax	$(B, 3)$	softmax regression head (4.8)
Experts	NEC_MLP $\times 2$ on $x[:, -1, :-1]$	$(B,)$ each	4-layer ReLU funnels (6.1–6.2)
Routing	where(argmax(prob) == 1, e, n)	$(B,)$	hard binary selection

The defect ledger, each entry with its diagnosing section:

1. *Hard argmax routing.* $\partial(\arg \max)/\partial(\text{logits}) = 0$ a.e.: the regression loss sends no gradient to the gate (B §B.8.4; Module 4 §4.10). The gate must be trained on a side objective that need not align with predictive performance.
2. *Three classes, one bit.* The router checks only `== 1`: classes 0 and 2 are collapsed, discarding a third of the computed posterior (B §B.8.4; the thesis syllabus’s founding observation).
3. *Explicit nn.Softmax inside the model.* The head emits probabilities, not logits. If trained with `CrossEntropyLoss` (which expects logits) this is simply wrong — softmax applied twice; if trained with a log of these probabilities, it is the unfused, numerically unstable chain that Module 4 §4.10 and Foundations B §B.6.3 exist to forbid. Corrected models emit logits and use the fused loss.
4. *BatchNorm in the classifier head.* Cross-sample coupling of dates inside each batch — the leakage channel of Module 6 §6.7’s box, sitting directly upstream of the regime decision. LayerNorm is the drop-in repair.
5. *Encoder scale.* Eight LSTM layers at $n = 256$ ($\sim 4\text{M}$ parameters, inter-layer dropout 0.4 fighting the consequences) against the effective-sample arithmetic of §7.5. A 1–2 layer GRU at $n \in [16, 64]$ is the right opening bid, by the parameter-count argument of Exercise 7.5.
6. *Sequence information reaches only the gate.* Both experts consume $x[:, -1, :-1]$ — the last step’s raw features. Defensible as a design (regime-from-history, prediction-from-snapshot), but it means the encoder’s entire value flows through the routing decision that defect 1 prevents from training. The corrected model should decide *deliberately* whether experts also see h_T , and say so in the thesis.

7. *Silent feature slicing.* `[: -1]` drops the last feature column undocumented — a column evidently reserved (a label? an ID?). Until provenance is established, this is a potential leakage or alignment bug of exactly the kind Module 5 §5.4 catalogues; the corrected pipeline names its columns.
8. *Default-config mismatch.* `C_L` defaults to $n = 256$ while `C_F` expects 64: the defaults do not compose — evidence the file was assembled from screenshots, and a reminder that the corrected implementation needs shape assertions (Module 6 §6.9, step 1).

NEC connection

What the corrected NEC keeps, changes, and decides — the chapter’s deliverable to the thesis. *Keep:* the architecture’s thesis — a sequence encoder informing a regime gate over specialized experts; it is a sound MoE skeleton (B §B.8). *Change* (each with its derivation): soft routing trained by the mixture NLL (B.8.3, gradients B.8.5–B.8.6) replacing defects 1–2; logits + fused loss replacing defect 3; LayerNorm replacing defect 4; a small GRU replacing defect 5; named, audited feature columns replacing defect 7; shape assertions for defect 8. *Decide and document:* experts on snapshot features vs. snapshot + h_T (defect 6) — run both under Module 5’s protocol, pre-registered. Every item on this list now has a textbook section behind it; none is a matter of taste.

Checkpoint (from the syllabus). Closed book: derive BPTT (7.2.1) for $T = 3$ and in general; prove the vanishing/exploding theorem and state the eigenvalue intuition honestly (Lemma 7.3.1’s role, the tanh saturation factor); derive the GRU and LSTM gradient highways (7.4.2)/(7.4.4) and say which gates do what; and explain, at the level of mathematics, what `C_L` and `C_F` compute in the current code and which textbook results indict each defect in the ledger. If all four are comfortable, Part III is complete: you have the math, the models, the judge, the optimizer, and the architecture — and the road into Part IV (mixtures, EM, and the thesis’s core) is open.

7.7 Exercises

Importance labels. Each exercise carries a priority badge for the NEC/MoE thesis track, reflecting how load-bearing it is — *not* how hard it is. [★★★ Core] mark what the checkpoint and the corrected encoder demand closed-book: BPTT (7.1), the vanishing/exploding theorem done honestly (7.2), the two gradient highways (7.3, 7.4), and the GRU-vs-LSTM accounting behind the capstone’s “shrink the encoder” recommendation (7.5). [★★ Worth it] a notch below: truncation’s bias and its design corollary (7.6), the linear-RNN-as-EMA-bank reading (7.7), the bidirectional-leakage adjudication (7.8), and the applied milestone (7.9).

Questions only, as established. **Core exercises ([★★★ Core]) sit inline in the chapter body as checkpoint exercises, at the end of the section or subsection they test;** this section collects the supplementary exercises only. The six syllabus analytical exercises all appear; the two assigned textbook-problem sets were invalid (see preface: Prince Ch. 12 covers transformers; Goodfellow has no exercises) and are replaced by two substitutes chosen to cover the same conceptual ground with this pipeline’s priorities. The applied milestones close the set, flagged per house policy.

On BPTT (Section 7.2)

Exercise 7.6 [Syllabus: truncated BPTT’s bias]. [★★ Worth it] *Truncated BPTT with window k keeps only the $t > T - k$ terms of (7.2.1). (a) Write the omitted remainder explicitly and show the truncated gradient is biased: its expectation misses exactly the long-range terms. (b) Under the uniform bound of Exercise 7.2(b), bound the remainder norm by a geometric tail $\propto \bar{\rho}^k$, and conclude: the bias is negligible iff $\bar{\rho}^k \ll 1$ — i.e. truncation is safe precisely when the long-range gradients were vanishing regardless. (c) Spell out the cruel corollary in one paragraph: for a model intended to learn long-horizon regime structure ($\bar{\rho} \approx 1$ on the gated highway), truncation at k caps the learnable horizon at roughly k steps no matter what the architecture protects — so window length T , truncation k , and the regime horizons of interest must be chosen together. What does this say about the NEC’s choice of T for 20–60-day regimes?*

Substitutes for the invalid textbook assignments (see preface)

Exercise 7.7 [Substitute for Prince Probs. 12.1–12.2: the linear RNN as a bank of EMAs]. [★★ Worth it] *Drop the nonlinearity: $\mathbf{h}_t = \mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t$, $\mathbf{h}_0 = \mathbf{0}$. (a) Solve the recursion: $\mathbf{h}_T = \sum_{t=1}^T \mathbf{W}^{T-t} \mathbf{U} \mathbf{x}_t$ — the encoder is a convolution of the input history with the matrix-power kernel $\mathbf{W}^{T-t}$. (b) Diagonalize $\mathbf{W}$ (assume diagonalizable, eigenvalues λ_j): show each eigen-coordinate of $\mathbf{h}_T$ is an exponentially-weighted sum of inputs with per-mode memory timescale $\tau_j = -1 / \log |\lambda_j|$, and that a one-dimensional case with $w = 1 - \alpha$ is exactly the standard EMA with smoothing α . (c) Complex eigenvalues: what input structure does a mode with $\lambda_j = re^{i\theta}$ detect? (Damped oscillation — give the period in days for $\theta = 2\pi/20$.) Why might that be useful or dangerous for a market-regime encoder? (d) Design question: an encoder meant to span reversal-to-momentum horizons of 5–60 trading days should have its $|\lambda_j|$ spread over what range? Compare with the OP pipeline’s hand-chosen rolling windows — what has the learned encoder actually been given license to tune?*

Exercise 7.8 [Substitute for Goodfellow Ex. 10.1–10.2: when is a bidirectional encoder lookahead?]. [★★ Worth it] *A bidirectional RNN runs a forward pass and a backward pass over the window and concatenates: $\mathbf{h}_t^{\text{bi}} = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$, where $\overleftarrow{\mathbf{h}}_t$ is a function of $\mathbf{x}_{t:T}$. (a) Show that any per-step output built on $\mathbf{h}_t^{\text{bi}}$ with $t < T$ depends on inputs dated after t . (b) Now adjudicate three usage patterns against Module 5 §5.4’s information-set discipline, assuming the window $[1, T]$ ends at the prediction date and the label lives at $T + h$: (i) per-step labels $\mathbf{y}_t$ predicted from $\mathbf{h}_t^{\text{bi}}$; (ii) a single prediction from $\mathbf{h}_T^{\text{bi}}$; (iii) a single prediction from the concatenated final states $[\vec{\mathbf{h}}_T; \overleftarrow{\mathbf{h}}_1]$. Which are leakage, which are legal, and why? (Careful: one of these is legal despite “using the backward pass,” and one is illegal despite looking innocuous.) (c) The existing NEC sets `bidirectional=False`. State in two sentences whether turning it on would be a bug or a choice for this architecture’s usage pattern, and what documentation requirement Module 5’s protocol would impose either way.*

Applied milestones

Exercise 7.9 [Applied milestone — optional, computational]. [★★ Worth it] *The syllabus’s two practical milestones, merged; the chapter’s one computational exercise, per house policy. (i)*

Implement a GRU cell from scratch in PyTorch (the four lines of (7.4.1) as tensor ops); validate forward outputs against `torch.nn.GRU` with copied weights, and gradient agreement under `autograd` on a toy sequence. (ii) Re-read `OP model/nec/nec_hybrid_explained.py` with the capstone's shape table in hand: verify every shape in the table by instrumenting a forward pass with random inputs, confirm the `C_L/C_F` default-dimension mismatch (defect 8) by triggering it, and write a three-line shape assertion block that would have caught it.

Source map and what comes next

Section	Primary source(s)	Feeds directly into
7.1 Encoders vs. seq2seq	GF §10.1–10.2, §10.4 (contrast)	NEC data interface
7.2 BPTT, truncation	GF §10.2.2 + Module 6 §6.4	encoder training mechanics
7.3 Vanishing/exploding	GF §10.7 + Fnd. C motif	window/horizon choices; clipping (6.6)
7.4 GRU, LSTM	GF §10.10; Olah §1–5	the corrected encoder
7.5 Hidden states, overfitting	synthesis + Module 5	encoder sizing; honesty tests
7.6 NEC code reading	the code + whole-book ledger	the corrected model's spec

Part III is complete. Part IV is the thesis's core: Math Foundations D (information theory deepened into variational analysis — the ELBO three ways, forward vs. reverse KL) and Module 8 (mixtures, EM, and variational inference), where the responsibilities of Foundations B §B.8, the weighted least squares of Module 4, and the gate gradients carried since (B.8.6) finally assemble into the algorithm the corrected NEC trains by. Every prerequisite it lists is now behind you.